

UNIT 9 · WEEK 15

AI-native OS: agents and LLMs in the operating system

📅 Week 15 · dates to be confirmed </> Lab TD8 (solved + AI challenge) ~ 4 animations

LAB TD8 · SOLVED (NEW)

An agentic OS with your own micro-LLM

Windows and Linux distributions are starting to host AI agents inside the system. This lab builds that architecture in miniature with everything in the course: a **micro-LLM** trained for the course turns requests in Spanish or English into a tool call; a **daemon** (`agentd`) receives it over a UNIX socket, validates it with a **policy**, asks for **consent** if it is destructive and runs it in a process locked in by **Landlock**, writing everything to an **audit log**.

</> Build and test

Bash

```
make -C tds                                # also builds td8: tinyllm, agentd, agent,
agent_exec
tds/tests/run.sh td8                        # 24 checks: model, requests, policy,
consent, Landlock, audit
cd tds/bin/td8 && ./tinyllm -v "list the files in docs"
```

It needs Linux ≥ 5.13 with Landlock enabled (`cat /sys/kernel/security/lsm`). Training is optional (the weights are included): `make -C tds/td8 train` (NumPy, ≈ 1 h on a CPU).

✓ Assignment: lab report

Hand in this lab with a short LaTeX report (just what is needed). What it must cover, the template, an example and the Prism guide are in the Assignment tab. Generative AI allowed, with a declaration.

EXERCISES

- | | | | |
|---|--|---|----------------------------|
| 1 | The micro-LLM: data and training | 4 | The policy and consent |
| 2 | Inference in C with <code>mmap</code> and a KV cache | 5 | Defence in depth: Landlock |
| 3 | <code>agentd</code> : socket, thread pool and identity | 6 | Auditing and reflection |

TD8 · 1

● Solved · tested

The micro-LLM: data and training

Statement. Read `train/dataset.py` and `train/train.py`. What exactly does the model learn? Why are file names generated at random? How do we know the hand-written gradient is correct?

Solution

Python `tds/td8/train/dataset.py`

```
"""Synthetic data for the TD8 micro-LLM: requests in Spanish or English → tool call.
```

```
Every example is a text of UTF-8 bytes:
```

```
Q: lista los archivos de docs\n
A: list_files("docs")\n
```

```
The model only learns to translate the intent into ONE call from a closed catalogue of tools (the equivalent of the MCP "connectors" of an agentic OS). File and folder names are generated at random so that the model learns to COPY them from the request (attention), not to memorise them. None of this is safe by itself: the OS decides whether to run the call (policy + Landlock in agentd).
```

- A GPT-style **byte-level** transformer (vocabulary 256, context 128, d = 64, 4 heads, 3 layers, ≈ 175 thousand parameters), trained with plain NumPy (no PyTorch) and

hand-written backpropagation; `python3 train.py --check` compares the analytical gradient with finite differences.

- It learns to translate a request into **one** call from a closed catalogue of 12 tools. Random file names force the model to **copy** them from the request (attention) instead of memorising them.
- **Real result** (`tds/td8/weights/train_log.json`): 174,656 parameters, 12,000 steps ($\approx$ 87 min on 4 CPU threads), **99.0%** exact match on 500 new requests; the hardest tool is `write_note` (89%), because two arguments must be copied. Weights: 698,688 bytes.
- **Data lesson:** the previous version (v1) scored 99.4% on its test, but in the lab it turned `viejo.txt` into `viefe.txt` and did not recognize `atajo`: it had seen few names like those. Instead of changing the network, the data generator was widened (names from the whole alphabet, without extension). The model is only as good as its data, and that is why `agentd` never trusts it.
- The data deliberately includes dangerous paths (`/etc/passwd`, `../..`): the model must copy them faithfully, because deciding whether they are acceptable **is not its job**.

See it inside in animation A9.2 →

TD8 · 2

● Solved · tested

Inference in C with `mmap` and a KV cache

Statement. Explain how `tinylm.c` loads the weights without copying them and why every new token costs a single pass through the network. Run `./tinylm -v` and relate its metrics to units 4 and 6. Check that C and NumPy compute the same thing.

Solution

```
C tds/td8/tinylm.c

int tl_load(tl_model *m, const char *path)
{
    memset(m, 0, sizeof *m);
    int fd = open(path, O_RDONLY);
    if (fd == -1)
        return -1;
    struct stat st;
```

```

if (fstat(fd, &st) == -1 || st.st_size < HEADER) {
    close(fd);
    errno = EINVAL;
    return -1;
}

/* MAP_PRIVATE + PROT_READ: nobody can modify the weights through this
mapping. */
void *map = mmap(NULL, (size_t)st.st_size, PROT_READ, MAP_PRIVATE, fd,
0);
close(fd); /* the mapping keeps the
file alive */
if (map == MAP_FAILED)
    return -1;

const unsigned char *raw = map;
int32_t h[7];
memcpy(h, raw + 8, sizeof h);
if (memcmp(raw, "FSOLLM01", 8) != 0 || h[0] != 1 || h[4] > TL_MAX_LAYERS
|| h[3] % h[5] != 0) {
    munmap(map, (size_t)st.st_size);
    errno = EINVAL;
    return -1;
}

m->vocab = h[1]; m->ctx = h[2]; m->d = h[3]; m->n_layer = h[4]; m->n_head
= h[5]; m->d_ff = h[6];
m->map = map;
m->map_size = (size_t)st.st_size;

/* The pointers point INTO the mapping: no copy (the file is little-
endian float32). */
const float *p = (const float *)(raw + HEADER);
const int d = m->d, f = m->d_ff;
#define TAKE(dst, n) do { (dst) = p; p += (n); } while (0)
TAKE(m->wte, (long)m->vocab * d);
TAKE(m->wpe, (long)m->ctx * d);
for (int l = 0; l < m->n_layer; l++) {
    tl_layer *L = &m->layer[l];
    TAKE(L->ln1_g, d); TAKE(L->ln1_b, d);
    TAKE(L->w_qkv, (long)d * 3 * d); TAKE(L->b_qkv, 3 * d);
    TAKE(L->w_o, (long)d * d); TAKE(L->b_o, d);
    TAKE(L->ln2_g, d); TAKE(L->ln2_b, d);
    TAKE(L->w_fc, (long)d * f); TAKE(L->b_fc, f);
    TAKE(L->w_pr, (long)f * d); TAKE(L->b_pr, d);
}
TAKE(m->lnf_g, d); TAKE(m->lnf_b, d);

```

```
#undef TAKE
    m→n_params = (long)(p - (const float*)(raw + HEADER));
    if ((size_t)HEADER + (size_t)m→n_params * sizeof(float) ≠ m→map_size)
    {
        munmap(map, m→map_size);
        errno = EINVAL;
        return -1;
    }
    return 0;
}
```

Real output

```
$ ./tinylm -v "lista los archivos de docs"
step 0: 'l' 100.0% 'f' 0.0% 'r' 0.0%
step 1: 'i' 100.0% 'q' 0.0% '4' 0.0%
step 2: 's' 100.0% 'g' 0.0% 'q' 0.0%
step 3: 't' 100.0% 'k' 0.0% 'b' 0.0%
step 4: '_' 100.0% '-' 0.0% 'k' 0.0%
step 5: 'f' 100.0% '2' 0.0% 'c' 0.0%
step 6: 'i' 100.0% 'o' 0.0% 'n' 0.0%
step 7: 'l' 100.0% 'L' 0.0% 's' 0.0%
step 8: 'e' 100.0% 'o' 0.0% 'f' 0.0%
step 9: 's' 100.0% '(' 0.0% 'q' 0.0%
step 10: '(' 100.0% ' ' 0.0% 'g' 0.0%
step 11: '"' 100.0% ' ' 0.0% 'R' 0.0%
step 12: 'd' 100.0% 'k' 0.0% 'b' 0.0%
step 13: 'o' 100.0% 'i' 0.0% 'e' 0.0%
step 14: 'c' 100.0% 'k' 0.0% 'f' 0.0%
step 15: 's' 100.0% 'q' 0.0% 'f' 0.0%
step 16: '"' 100.0% 'z' 0.0% 'k' 0.0%
step 17: ')' 100.0% ',' 0.0% ':' 0.0%
step 18: '\n' 100.0% 'A' 0.0% ' ' 0.0%
```

```
list_files("docs")
```

```
model      : 174656 parameters, 3 layers, 4 heads, d=64, context 128 bytes
weights    : 698688 bytes mapped with mmap (MAP_PRIVATE, PROT_READ)
KV cache   : 201984 bytes per request (+ scratch)
tokens     : 33 of prompt (prefill) + 19 generated (decode)
time       : load 0.042 ms, inference 9.08 ms (5730 tokens/s)
page faults: 2 during load, 47 during the first inference (minor; major: 0)
```

Real output

```
$ python3 train/compare.py ./tinyllm 200
200/200 respuestas idénticas entre NumPy y C
```

`mmap` with `PROT_READ` and `MAP_PRIVATE` returns an address; the pointers of every layer point *inside* the mapped file. Nothing is read until the first token touches the weights: that is why page faults show up during the first inference and not during loading. The KV cache stores, per layer and per position, the keys and values already computed: every new byte only computes its own row. The C and NumPy inference give **the same answers** on 200 new requests.

TD8 · 3

● Solved · tested

agentd : socket, thread pool and identity

Statement. Start `agentd` with a workspace and send it requests with `agent`. Identify in the code which part comes from each course unit. How does `agentd` know who is talking to it?

Solution

C tds/td8/agentd.c

```
static void serve(int client)
{
    struct ucred cred = { 0 };
    socklen_t len = sizeof cred;
    if (getsockopt(client, SOL_SOCKET, SO_PEERCREDS, &cred, &len) == -1)
        return;

    char req[MAX_REQ], out[256], why[PATH_MAX + 64];
    if (read_line(client, req, sizeof req) == 0)
        return;

    /* 1. The model proposes. */
    int steps = tl_generate(&model, req, out, sizeof out, NULL, NULL);
    if (steps < 0) {
        send_str(client, "error: request too long\n");
        audit(cred.uid, cred.pid, req, "-", "deny", "too long");
        return;
    }
}
```

```

send_fmt(client, "model: %s\n", out);

/* 2. The operating system disposes. */
call_t c;
if (parse_call(out, &c) == -1) {
    send_str(client, "policy: deny (the model produced an invalid
call)\n");
    audit(cred.uid, cred.pid, req, out, "deny", "invalid call");
    return;
}
decision_t d = policy(&c, why, sizeof why);
send_fmt(client, "policy: %s (%s)\n", DECISION[d], why);
if (d == DENY) {
    audit(cred.uid, cred.pid, req, out, "deny", why);
    return;
}
if (d == ASK) {
    char answer[16];
    send_fmt(client, "ask: allow the agent to run %s? [y/N]\n", out);
    read_line(client, answer, sizeof answer);
    if (answer[0] != 'y' && answer[0] != 'Y') {
        send_str(client, "cancelled by the user\n");
        audit(cred.uid, cred.pid, req, out, "ask", "refused by the
user");
        return;
    }
}

/* 3. Sandboxed execution. */
const char *outcome = run_tool(client, &c);
audit(cred.uid, cred.pid, req, out, d == ASK ? "ask:approved" : "allow",
outcome);
}

```

Real output

```

$ ./agent "lista los archivos de docs"
model: list_files("docs")
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
2 entries in docs
-      0  plan.md
-      0  syllabus.pdf

```

Real output

```
$ ./agent "muéstrame notas.txt"
model: read_file("notas.txt")
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
Laboratorio TD8
fork, exec, pipe, mmap, sockets
```

Real output

```
$ ./agent "busca 'error' en log.txt"
model: search("error", "log.txt")
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
log.txt:2: error: disk full
log.txt:4: error: timeout
2 matching line(s) for "error"
```

Real output

```
$ ./agent "¿qué versión del núcleo tengo?"
model: system_info()
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
Linux 7.0.0-34-generic (x86_64), 28 CPUs
memory: 32.8 GB total, 6.9 GB free; 3110 processes; up 4 h 7 min
```

Real output

```
$ for i in $(seq 8); do ./agent "lista mis archivos" & done; wait
8
```

AGENTD PIECE	UNIT
UNIX socket with 0600 permissions	U8 · U3
<code>SO_PEERCRED</code> : client uid and pid, given by the kernel	U8
Thread pool with a bounded queue (semaphores + mutex)	U7
Weights shared with <code>mmap</code>	U6
<code>fork</code> + <code>execv</code> of the executor; output through a pipe	U4 · U5
<code>SIGTERM</code> to stop; <code>SIGPIPE</code> ignored	U6

TD8 · 4

● Solved · tested

The policy and consent

Statement. Ask the agent to delete a workspace file (answering yes and no), to delete `/etc/passwd`, to read `../../etc/shadow` and to read `atajo`, a symbolic link to `/etc/passwd` inside the workspace. Who decides in each case: the model or the OS?

Solution



C tds/td8/agentd.c

```
static int inside_workspace(const char *rel, char *why, size_t whysz)
{
    if (rel[0] == '/' || rel[0] == '~') {
        snprintf(why, whysz, "absolute path outside the workspace");
        return 0;
    }
    if (strcmp(rel, "..") == 0 || strncmp(rel, "../", 3) == 0 || strstr(rel,
"/../") || strstr(rel, "/..") ) {
        snprintf(why, whysz, "'..' leaves the workspace");
        return 0;
    }
    /* Resolve symbolic links: the file (or, if it does not exist yet, its
directory). */
    char full[PATH_MAX * 2], real[PATH_MAX];
    snprintf(full, sizeof full, "%s/%s", workspace, rel);
    if (realpath(full, real) == NULL) {
        char *slash = strrchr(full, '/');
        *slash = '\0';
        if (realpath(full, real) == NULL) {
            snprintf(why, whysz, "directory does not exist");
            return 0;
        }
    }
    size_t n = strlen(workspace);
    if (strncmp(real, workspace, n) != 0 || (real[n] != '\0' && real[n] !=
'/')) {
        snprintf(why, whysz, "resolves to %s, outside the workspace", real);
        return 0;
    }
    return 1;
}

static decision_t policy(const call_t *c, char *why, size_t whysz)
{
    const tool_t *t = c->tool;
    if (strcmp(t->name, "none") == 0) {
        snprintf(why, whysz, "no tool for this request");
        return DENY;
    }
    if (policy_on && t->path_arg ≥ 0 && !inside_workspace(c->args[t
->path_arg], why, whysz))
        return DENY;
    if (t->consent) {
```

```
    snprintf(why, whysz, "destructive action: needs the user's consent");
    return ASK;
}
    snprintf(why, whysz, policy_on ? "tool in the catalogue, path inside the
workspace" : "POLICY DISABLED");
    return ALLOW;
}
```

Real output

```
$ ./agent -n "borra viejo.txt"
model: delete_file("viejo.txt")
policy: ask (destructive action: needs the user's consent)
ask: allow the agent to run delete_file("viejo.txt")? [y/N]
> n (automatic answer)
cancelled by the user
```

Real output

```
$ ./agent -y "borra viejo.txt"
model: delete_file("viejo.txt")
policy: ask (destructive action: needs the user's consent)
ask: allow the agent to run delete_file("viejo.txt")? [y/N]
> y (automatic answer)
[sandbox: Landlock ABI 8, workspace only]
viejo.txt deleted
```

Real output

```
$ ./agent "borra /etc/passwd"
model: delete_file("/etc/passwd")
policy: deny (absolute path outside the workspace)
```

Real output

```
$ ./agent "lee ../../etc/shadow"
model: read_file("../../etc/shadow")
policy: deny ('..' leaves the workspace)
```

Real output

```
$ ./agent "lee atajo"
model: read_file("atajo")
policy: deny (resolves to /etc/passwd, outside the workspace)
```

Real output

```
$ ./agent "cuéntame un chiste"
model: none()
policy: deny (no tool for this request)
```

The model always proposes what it was asked —including `delete_file("/etc/passwd")` —; the one that decides is `agentd`: absolute paths, `..` and escaping links (resolved with `realpath`) are denied; anything destructive asks the user.

Try it with your own requests in animation A9.4 →

TD8 · 5

● Solved · tested

Defence in depth: Landlock

Statement. Restart `agentd` with `--no-policy` (simulating a bug in the policy) and repeat the attacks. Then run `agent_exec` directly with and without `--no-sandbox`. Which layer stops you now?

Solution

C tds/td8/agent_exec.c

```
static int sandbox(const char *workspace)
{
    int abi = ll_create(NULL, 0, LANDLOCK_CREATE_RULESET_VERSION);
    if (abi < 1)
        return 0;

    __u64 fs = LANDLOCK_ACCESS_FS_EXECUTE | LANDLOCK_ACCESS_FS_WRITE_FILE |
LANDLOCK_ACCESS_FS_READ_FILE |
LANDLOCK_ACCESS_FS_READ_DIR | LANDLOCK_ACCESS_FS_REMOVE_DIR |
```

```

LANDLOCK_ACCESS_FS_REMOVE_FILE |
    LANDLOCK_ACCESS_FS_MAKE_CHAR | LANDLOCK_ACCESS_FS_MAKE_DIR |
LANDLOCK_ACCESS_FS_MAKE_REG |
    LANDLOCK_ACCESS_FS_MAKE_SOCKET | LANDLOCK_ACCESS_FS_MAKE_FIFO |
LANDLOCK_ACCESS_FS_MAKE_BLOCK |
    LANDLOCK_ACCESS_FS_MAKE_SYM;
if (abi ≥ 2) fs |= LANDLOCK_ACCESS_FS_REFER;
if (abi ≥ 3) fs |= LANDLOCK_ACCESS_FS_TRUNCATE;
if (abi ≥ 5) fs |= LANDLOCK_ACCESS_FS_IOCTL_DEV;

struct landlock_ruleset_attr attr = { .handled_access_fs = fs };
size_t size = sizeof(__u64); /* ABI 1-3:
only the fs field */
if (abi ≥ 4) {
    attr.handled_access_net = LANDLOCK_ACCESS_NET_BIND_TCP |
LANDLOCK_ACCESS_NET_CONNECT_TCP;
    size = offsetof(struct landlock_ruleset_attr, handled_access_net) +
sizeof(__u64);
}
if (abi ≥ 6) {
    attr.scoped = LANDLOCK_SCOPE_ABSTRACT_UNIX_SOCKET |
LANDLOCK_SCOPE_SIGNAL;
    size = sizeof attr;
}
int rs = ll_create(&attr, size, 0);
exit_if(rs == -1, "landlock_create_ruleset");

/* Everything inside the workspace (minus executing programs), read-only
/proc. */
exit_if(ll_add_path(rs, workspace, fs & ~
(__u64)LANDLOCK_ACCESS_FS_EXECUTE) == -1, "landlock_add_rule workspace");
exit_if(ll_add_path(rs, "/proc", LANDLOCK_ACCESS_FS_READ_FILE |
LANDLOCK_ACCESS_FS_READ_DIR) == -1,
"landlock_add_rule /proc");

exit_if(prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0) == -1, "prctl");
exit_if(syscall(SYS_landlock_restrict_self, rs, 0) == -1,
"landlock_restrict_self");
close(rs);
return abi;
}

static void limits(void)
{
    struct rlimit cpu = { 2, 3 }, fsize = { 1 << 20, 1 << 20 }, nofile = {

```

```
64, 64 };
    exit_if(setrlimit(RLIMIT_CPU, &cpu) == -1, "setrlimit cpu");      /*
SIGXCPU after 2 s */
    exit_if(setrlimit(RLIMIT_FSIZE, &fsize) == -1, "setrlimit fsize"); /*
SIGXFSZ past 1 MiB */
    exit_if(setrlimit(RLIMIT_NOFILE, &nofile) == -1, "setrlimit nofile");
    alarm(5);                                                         /*
SIGALRM: real time */
}
```

Real output

```
$ ./agent "lee atajo"
model: read_file("atajo")
policy: allow (POLICY DISABLED)
[sandbox: Landlock ABI 8, workspace only]
read_file: atajo: Permission denied
```

Real output

```
$ ./agent "lee /etc/passwd"
model: read_file("/etc/passwd")
policy: allow (POLICY DISABLED)
[sandbox: Landlock ABI 8, workspace only]
read_file: /etc/passwd: Permission denied
```

Real output

```
$ ./agent_exec workspace read_file /etc/hostname; ./agent_exec --no-sandbox
workspace read_file /etc/hostname
[sandbox: Landlock ABI 8, workspace only]
read_file: /etc/hostname: Permission denied
[sandbox: DISABLED]
host
```

Before touching anything, `agent_exec` locks itself in: limits on CPU, file size and descriptors (`setrlimit`), a 5 s alarm, `no_new_privs` and a Landlock rule that only allows the workspace (and `/proc` read-only), denies TCP connections and signals outside the sandbox. Even with the policy disabled, the **kernel** answers `Permission denied`. The same program, without the sandbox, does read the file.

TD8 · 6

● Solved · tested

Auditing and reflection

Statement. Read `audit.log`. What information lets you reconstruct what the agent did and on whose behalf? What does this agentic OS lack to be used for real?

Solution



Real output

```

2026-09-30T13:14:12 uid=1000 pid=364726 request="lista los archivos de docs"
call=list_files("docs") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364735 request="muéstrame notas.txt"
call=read_file("notas.txt") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364744 request="¿cuántas líneas tiene
main.c?" call=count_lines("main.c") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364753 request="busca 'error' en log.txt"
call=search("error", "log.txt") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364762 request="anota 'examen el lunes' en
agenda.md" call=write_note("agenda.md", "examen el lunes") decision=allow
outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364771 request="crea la carpeta proyectos"
call=make_dir("proyectos") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364780 request="¿cuánto espacio libre
queda?" call=disk_usage() decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364789 request="¿qué versión del núcleo
tengo?" call=system_info() decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364798 request="show running processes"
call=processes() decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364807 request="borra viejo.txt"
call=delete_file("viejo.txt") decision=ask outcome=refused by the user
2026-09-30T13:14:13 uid=1000 pid=364815 request="borra viejo.txt"
call=delete_file("viejo.txt") decision=ask:approved outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364824 request="borra /etc/passwd"
call=delete_file("/etc/passwd") decision=deny outcome=absolute path outside
the workspace
2026-09-30T13:14:13 uid=1000 pid=364832 request="lee ../../etc/shadow"
call=read_file("../../etc/shadow") decision=deny outcome='..' leaves the
workspace
2026-09-30T13:14:13 uid=1000 pid=364840 request="lee atajo"
call=read_file("atajo") decision=deny outcome=resolves to /etc/passwd,
outside the workspace
2026-09-30T13:14:13 uid=1000 pid=364848 request="cuéntame un chiste"
call=none() decision=deny outcome=no tool for this request
2026-09-30T13:14:13 uid=1000 pid=364873 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364875 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364877 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364876 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364880 request="lista mis archivos"

```

```

call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364879 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364881 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364878 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok

```

- Every line has the date, the client's uid and pid (given by the kernel, not by the client), the request, the proposed call, the decision and the outcome: it tells what the agent did apart from what the user asked, as Windows requires with its agent accounts.
- It lacks, among other things: its own user for the agent (today it runs as the instructor), a `seccomp` filter, memory limits with cgroups, protection of the log against the agent itself and a defence against prompt injection once the agent reads file contents.

Checklist

Generative AI challenge · A new tool for the agent... and an attack



Goal. Add a `copy_file(source, destination)` tool to the agentic OS: generate data, retrain the micro-LLM, extend the policy and the executor. Then try to make the agent copy a file out of the workspace and check which layer stops it.

Starting prompt. Use it in the tool you prefer (Claude, ChatGPT, Gemini, Copilot, Codex...) with the *solved* version of the lab as context. Ask for a plan first and review every step before accepting it.

Prompt

I am working with a toy agentic OS in C: a micro-LLM (tinyllm.c) turns requests into tool calls, agentd applies a policy and agent_exec runs the tool inside a Landlock sandbox [paste dataset.py, the TOOLS table of agentd.c and main() of agent_exec.c]. I want to add `copy_file(source, destination)`. Guide me to: (1) add Spanish and English templates to dataset.py; (2) retrain and measure the accuracy before and after; (3) extend the policy so that BOTH arguments stay inside the workspace; (4) implement the copy in agent_exec. Then help me design 5 malicious requests (absolute paths, '..', symbolic links, injection in the text) and predict which layer stops each one: the parser, the policy or Landlock.

How to check the result

- The accuracy of the retrained model on the previous tools does not drop by more than 2 points (`train.py --eval`).
- `copy_file` with a destination outside the workspace ends in `policy: deny`, and with `--no-policy` in `Permission denied` (Landlock).
- The `audit.log` file records the 5 attempts with their decision.

Declare your use of AI: tool, relevant prompts, what you checked yourself and what you fixed. What counts is that you can explain and verify every decision.